

Ministry of Education of Republic of Moldova
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics

Laboratory work 3: The Command Line: Shell, Pipes, Text Filters

Author:
st. gr. FAF-241 Mitrofan Alexia
Verified:
asist. univ. Reițman Patricia

Chișinău - 2026

Contents

OBJECTIVES **2**

Before the start **2**

Part 1. Moving fast **3**

Part 2. The shell working **7**

Part 3. Redirection, chaining and jobs **11**

Part 4. Answering questions with one liners **13**

Part 5. First script **16**

CONCLUSION **18**

REFERENCES **20**

OBJECTIVES

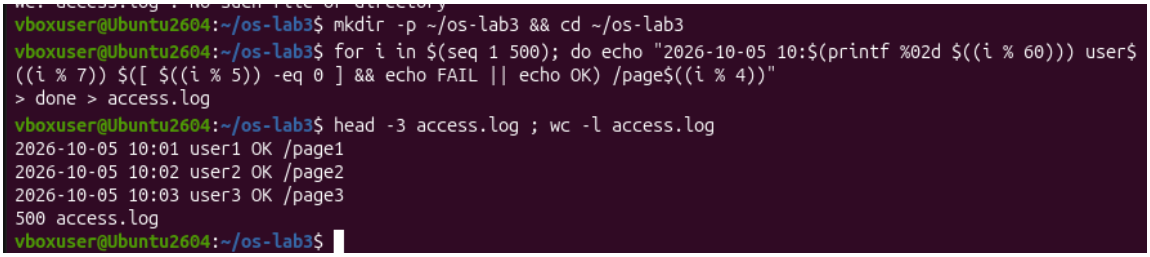
The goal of this laboratory work is to learn what bash commands do and be able to work fast with them, predict what a shell does to a command line before it runs, redirect output where we want it to, and answer real questions about data with one line of small filters joined by pipes.

Before the start

Here we set up the os2026-setup we received, which compiles the xv6-riscv program, and also we create the `access.log` file with the specifications on line 3.

```
1 bash os2026-setup.sh
```

```
1 mkdir -p ~/os-lab3 && cd ~/os-lab3
2 for i in $(seq 1 500); do
3 echo "2026-10-05 10:$(printf %02d $((i % 60))) user$((i % 7)) $([ $((i
    % 5)) -eq 0 ]  && echo FAIL || echo OK) /page$((i % 4))"
4 done > access.log
5 head -3 access.log ; wc -l access.log
```



Creating access.log file

Part 1. Moving fast

```
ls /usr/sh
```



Figure 1.1: Result of pressing tab twice after typing "ls /usr/sh"

```
cd /et
```



Figure 1.2: Result of pressing tab once after typing "cd /et"

After typing any three random commands, pressing Ctrl+R, typing a few letters of the first command and pressing Enter, I noticed that we enter a sort of "(reverse-i-search)`"" menu, and typing a keyword unique to the first command (it was `cd /xv6-riscv`), I got the full command. Pressing Enter ran the command, even without fully typing it.

```
type cd ; type ls ; type echo ; which ls
echo $PATH
echo $PATH | tr ':' '\n'
# one directory per line
nosuchprog ; echo "exit code: $?"
```

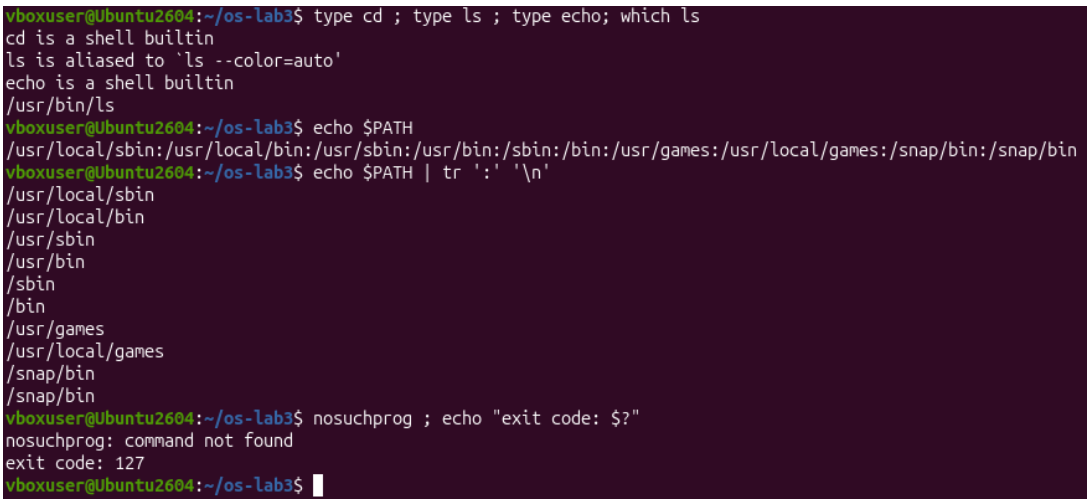


Figure 1.3: Built-in or external commands, PATH directories, exit code 127

```
man ls          # inside: /sort then n for next, q to quit
ls --help | head
man -k "list directory"
help cd
```

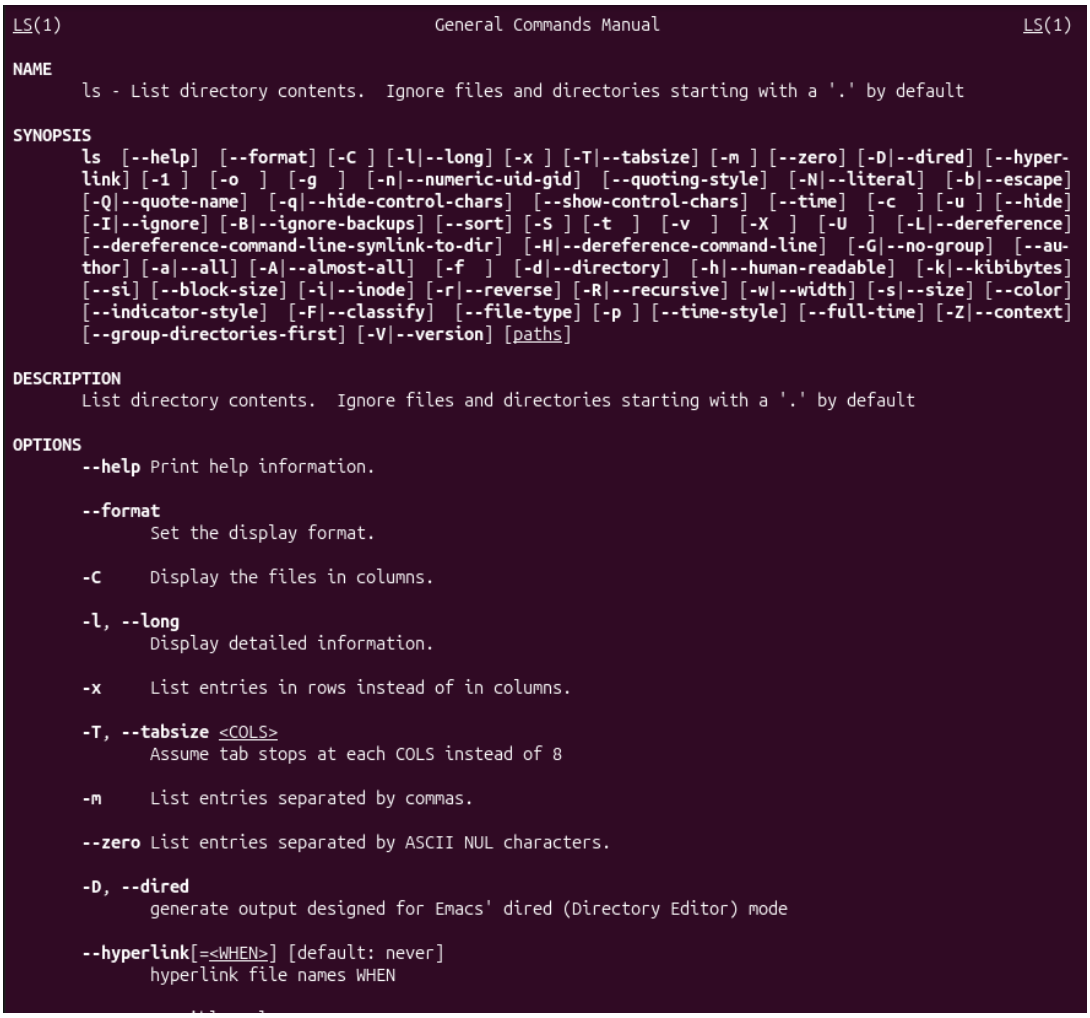


Figure 1.4: Finding help without a browser: man ls

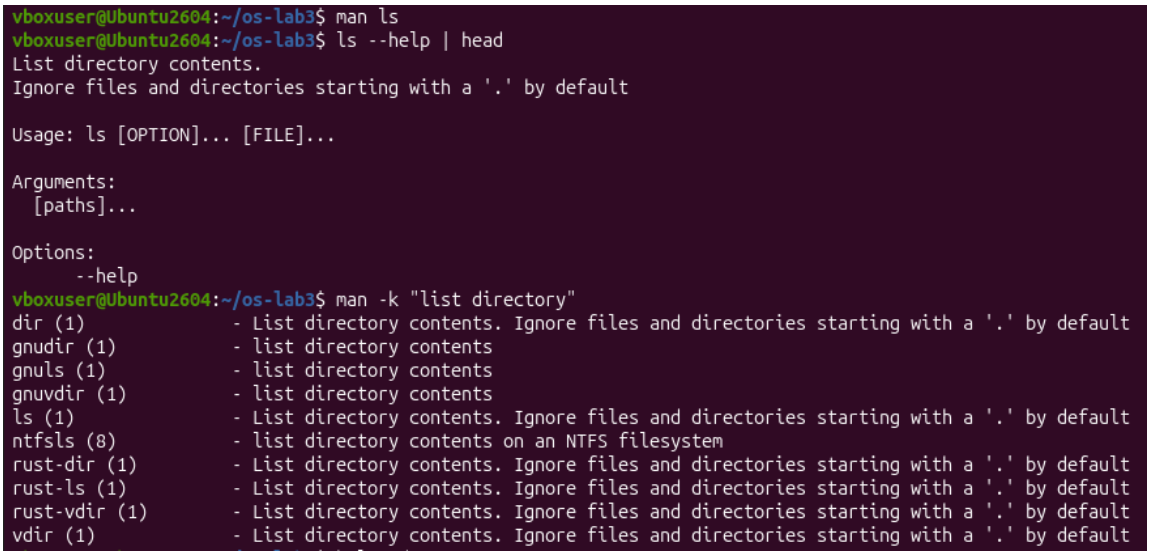


Figure 1.5: Finding help without a browser: ls −−help | head, man -k "list directory"

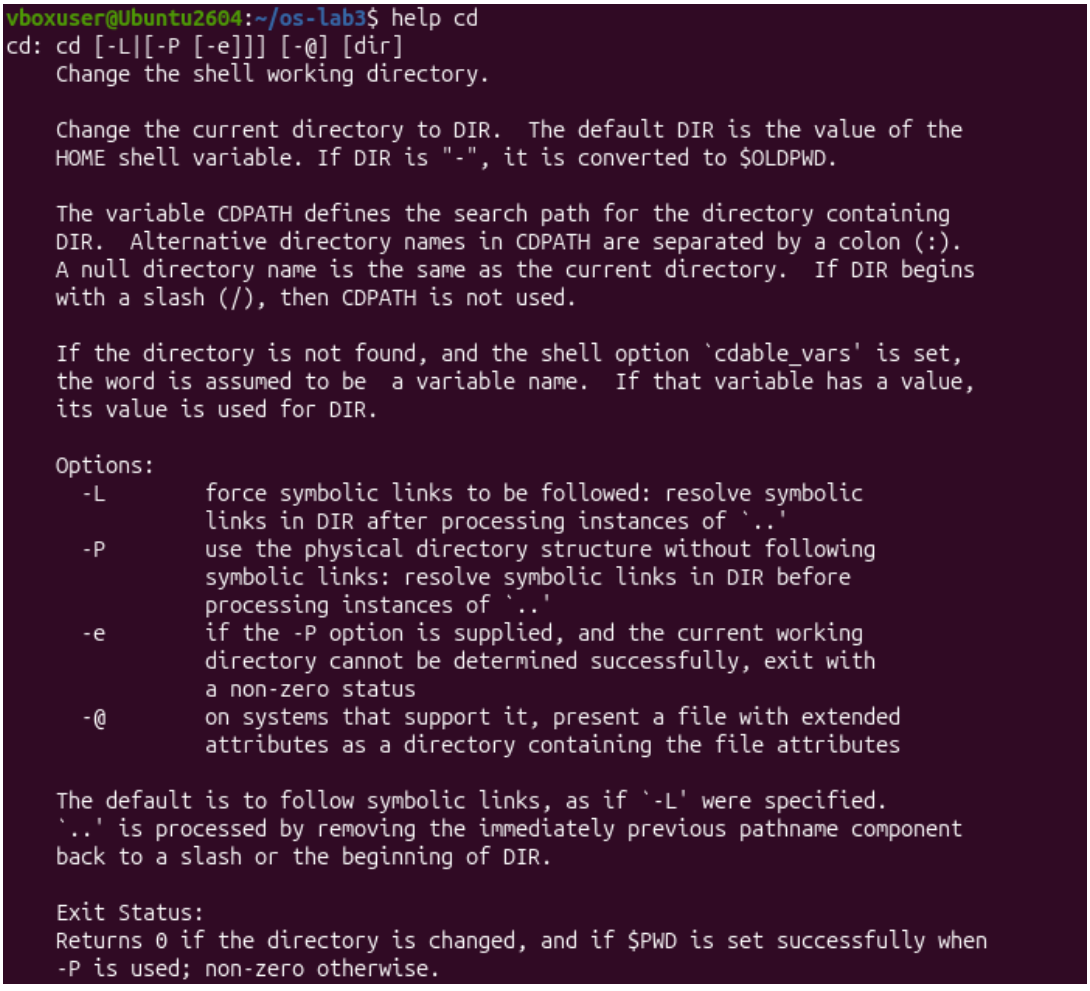


Figure 1.6: Finding help without a browser: help cd

1. **Which of cd, ls, echo are built-ins, and why MUST cd be one?**
   Out of these, `cd` and `echo` are built-ins. `cd` must be builtin because of how processes work in Unix: if `cd` were an external program, when typing `cd /directory` for example, the shell would fork a child, the child would call `chdir("/directory")` and exit. Basically the command would run successfully but we'd still be in the same directory. So the shell itself has to call the command `chdir`, which is why `cd` is builtin.

2. **How many directories are in your PATH, and in which one is ls?**
   There are 10 entries, but the last one repeats twice, so there are 9 distinct paths. `ls` is inside `usr/bin/ls`.

3. **What exit code does a command that is not found return?**
   The exit code is 127. This is basically the system not knowing what to do because the command was not found in PATH and is not a builtin shell command.

4. **Which option of ls sorts by modification time? (find it in man ls)**
   In `man ls`, the following option sorts by modification time: -t Sort by modification time (the 'mtime' in the inode), newest first.

Part 2. The shell working

```
mkdir -p ~/os-lab3/play && cd ~/os-lab3/play
touch a.c b.c file1.txt file2.txt notes.txt "alfa beta.txt"
ls
```

```
vboxuser@Ubuntu2604:~/os-lab3$ mkdir -p ~/os-lab3/play && cd ~/os-lab3/play
vboxuser@Ubuntu2604:~/os-lab3/play$ touch a.c b.c file1.txt file2.txt notes.txt "alfa beta.txt"
vboxuser@Ubuntu2604:~/os-lab3/play$ ls
 a.c  'alfa beta.txt'   b.c   file1.txt   file2.txt   notes.txt
```

Figure 2.1: Creating test directory and its files

```
echo *.c
echo file?.txt echo [a-b]* echo *.xyz
ls alfa beta.txt ls "alfa beta.txt"
echo '$HOME' "$HOME" \$HOME echo {1..5} ; echo $((7 * 6))
echo "today is $(date +%A), I am $(whoami)"
touch log{1..3}.txt && ls log*
```

```
vboxuser@Ubuntu2604:~/os-lab3/play$ echo *.c
a.c b.c
vboxuser@Ubuntu2604:~/os-lab3/play$ echo file?.txt
file1.txt file2.txt
vboxuser@Ubuntu2604:~/os-lab3/play$ echo [a-b]*
a.c alfa beta.txt b.c
vboxuser@Ubuntu2604:~/os-lab3/play$ echo *.xyz
*.xyz
vboxuser@Ubuntu2604:~/os-lab3/play$ ls alfa beta.txt
ls: cannot access 'alfa': No such file or directory
ls: cannot access 'beta.txt': No such file or directory
vboxuser@Ubuntu2604:~/os-lab3/play$ ls "alfa beta.txt"
'alfa beta.txt'
vboxuser@Ubuntu2604:~/os-lab3/play$ echo '$HOME' "$HOME" \$HOME
$HOME /home/vboxuser $HOME
vboxuser@Ubuntu2604:~/os-lab3/play$ echo {1..5} ; echo $((7 * 6))
1 2 3 4 5
42
vboxuser@Ubuntu2604:~/os-lab3/play$ echo "today is $(date +%A), I am $(whoami)"
today is Tuesday, I am vboxuser
vboxuser@Ubuntu2604:~/os-lab3/play$ touch log{1..3}.txt && ls log*
log1.txt  log2.txt  log3.txt
vboxuser@Ubuntu2604:~/os-lab3/play$
```

Figure 2.2: Glob pattern examples

Command	My prediction	Real output	Explanation
`echo *.c`	all .c files from the folder will be shown	all .c files from the folder were shown	* means everything, .c is the extension for c files
`echo file?.txt`	it will show all .txt files	it showed `file1.txt` `file2.txt`	the ? is a glob pattern, and it will match all the files that have "file" and a single additional character (can be any) in their name
`echo [a-b]*`	it will show all files that begin with a and b in alphabetical order	`a.c alfa beta.txt b.c` where shown, what was expected	[a-b]* is a glob pattern where [a-b] matches one character: a or b, and * matches the rest of the name
`echo *.xyz`	it will give error because there is no file with *.xyz extension	it simply wrote in terminal .xyz	the command `echo` doesn't care about files, it simply copied what we wrote and showed it in the terminal
`ls alfa beta.txt`	nothing happens, we are shown an error	the "no such files or directory" error was shown, since we didn't use quotes for the file name which contains a space	without quotes or spaces, ls receives two names, "alfa" and "beta.txt", neither of which exist
`ls "alfa beta.txt"`	it shows the content of the `alfa beta.txt` file, but since it's empty, nothing was shown	it looked for files with the exact name "alfa beta.txt"	I thought it would show the content inside the specific file, but instead it looked for files that matched the name exactly inside the directory

Command	My prediction	Real output	Explanation
`echo ‘$HOME$‘ "$HOME$" \HOME`	it will show our home directory 3 times: `/home/vboxuser`	the home directory was shown only once, in the argument written with double quotes	single quotes don't accept special symbols or variables, so anything written inside them will come out exactly as it is, and the backslash escapes the $, making it no longer special
`echo {1..5} ; echo $((7 * 6))`	shows numbers from 1 to 5, and then 42	it expanded the argument {1..5} to show the numbers from 1 to 5, and on the next line it showed 42	first is brace expansion, and second is arithmetic expansion
`echo "today is $(date +%A/), I am $(whoami)"`	it shows the same sentence, with the current day and user shown	it was shown exactly the same as in prediction	`$(...)` is modern command substitution, an alternative to backticks
`touch log{1..3}.txt && ls log*`	creates log1, log2 and log3 files and then shows them in the terminal	files `log1.txt`, `log2.txt` and `log3.txt` were created and displayed in the terminal	`log{1..3}` expanded into `log1.txt log2.txt log3.txt`, and since this command was successful, the second half was ran which displayed all files containing "log" in name and exactly one additional character

1. **echo knows nothing about files, yet echo *.c printed file names. Who produced them?**
   The shell produced them: first the `*.c` argument was "analyzed" by bash, which is a glob/pathname expansion, and it looked for all the files that match the pattern (any `.c` files from the working folder). The files were`a.c b.c`, and only after they were found `echo` was run. Essentially, to `echo` it's as if we passed `a.c b.c` instead of `*.c`.

2. **Why did ls alfa beta.txt fail? Give two different ways to fix it.**
   It failed because the full name contains a space, and since we didn't use quotes, bash receives "alfa" as an argument, which it doesn't understand. In order to fix it, we can use double or single quotes between the argument, or we can escape the space with a backslash: `ls alfa\ beta.txt`.

3. **What is the difference between single and double quotes?**
   Double quotes will interpolate things like: variables, backticks, certain \escapes, etc., but single quotes won't. Dollar signs, spaces, ampersands, asterisks and other special characters are all ignored when enclosed within single quotes.

Part 3. Redirection, chaining and jobs

```
cd ~/os-lab3
ls /etc /nope > out.txt 2> err.txt wc -l out.txt ; cat err.txt
ls /etc /nope > all.txt 2>&1 ; tail -2 all.txt
echo first > notes.md ; echo second >> notes.md ; cat notes.md wc -l <
    access.log
ls /nope 2> /dev/null || echo "failed with code $?"
mkdir -p tmp && cd tmp && echo "I am in $(pwd)" ; cd ~/os-lab3 sleep
    100 &
jobs ; kill %1 ; jobs
```

```
vboxuser@Ubuntu2604:~/os-lab3/play$ cd ..
vboxuser@Ubuntu2604:~/os-lab3$ ls /etc /nope > out.txt 2> err.txt
vboxuser@Ubuntu2604:~/os-lab3$ wc -l out.txt ; cat err.txt
239 out.txt
ls: cannot access '/nope': No such file or directory
vboxuser@Ubuntu2604:~/os-lab3$ ls /etc /nope > all.txt 2>&1 ; tail -2 all.txt
xml
zsh_command_not_found
vboxuser@Ubuntu2604:~/os-lab3$ echo first > notes.md ; echo second >> notes.md ; cat notes.md
first
second
vboxuser@Ubuntu2604:~/os-lab3$ wc -l < access.log
500
vboxuser@Ubuntu2604:~/os-lab3$ ls /nope 2> /dev/null || echo "failed with code $?"
failed with code 2
vboxuser@Ubuntu2604:~/os-lab3$ mkdir -p tmp && cd tmp && echo "I am in $(pwd)" ; cd ~/os-lab3
I am in /home/vboxuser/os-lab3/tmp
vboxuser@Ubuntu2604:~/os-lab3$ sleep 100 &
[1] 6415
vboxuser@Ubuntu2604:~/os-lab3$ jobs ; kill %1 ; jobs
[1]+  Running                 sleep 100 &
[1]+  Running                 sleep 100 &
[1]+  Terminated              sleep 100
vboxuser@Ubuntu2604:~/os-lab3$
```

Figure 3.1: Seeing different errors

```
ls /etc /nope > all.txt 2>&1
ls /etc /nope 2>&1 > all.txt
```

```
vboxuser@Ubuntu2604:~/os-lab3$ ls /etc /nope > all.txt 2>&1
vboxuser@Ubuntu2604:~/os-lab3$ ls /etc /nope 2>&1 > all.txt
ls: cannot access '/nope': No such file or directory
```

Figure 3.2: ls errors

1. **Which file got the error message, and which got the listing?**
   In figure 3.1, `err.txt` got the error message (`2> err.txt` redirects stderr), and `out.txt` got the listing(`> out.txt` redirects stdout).

2. **What is the difference between > all.txt 2>&1 and 2>&1 > all.txt ? Try both and explain why the result differs.**
   The main difference is the order of the directions. Bash processes them left to right. In the first case, > `all.txt` points stdout (fd\file descriptor 1) at `all.txt`, and `2>$1` points stderror (fd 2) at the same place stdout points, aka `all.txt`. Both streams end up in `all.txt`, so nothing appears on screen. In the second case, `2>$1` points stderr in the same place as stdout, which is `all.txt` the terminal, and then `>all.txt` points stdout at `all.txt`. Because stderr was already copied to the terminal before stdout moved, the error message still appears on screen and just the listing goes into `all.txt`, aka the ls. So the ls cannot access directory '/nope'... line.

3. **Why does wc -l < access.log print only a number, while wc -l access.log also prints the file name?**
   In `wc -l access.log`, the file name is given as an argument, so `wc` opens the file itself and prints the name next to the count (500 access.log). In `wc -l < access.log`, the shell opens the file and transmits its contents to `wc` through standard input. `wc` never receives the file name, so it only prints the number of lines (500).

4. **Explain each operator in one sentence: ; && ||.**
   The ; is a command separator, so it runs the next command from the same line regardless if the previous one succeeded or not. && acts like a logical AND, but instead of truth values it uses exit commands (0 for success, anything else for failure), so it runs the next command only if the previous one has an exit code of 0. || acts as a logical OR, and it runs the next command, regardless if the previous one succeeded or not.

Part 4. Answering questions with one liners

```
# Q1. How many requests failed?
grep FAIL access.log | cut -d' ' -f4 | wc -w
# Q2. How many different pages were requested, and which?
awk '{print $5}' access.log | sort -u | awk '{pages = pages " " $1} END {print NR ":" pages}'
# Q3. How many requests did each page get?
awk -F' ' '{print $5}' access.log | sort | uniq -c
# Q4. Which user or users have the most failed requests, and how many?
awk '$4 != "OK" {print $3, $4}' access.log | sort | uniq -c | sort -rn
# Q5. How many requests did user3 make, and how many of them failed?
awk '$3 == "user3" {print $3, $4}' access.log | sort | uniq -c | sort -rn | awk '{print $0; sum += $1} END {print "Total requests:", sum}'
# Q6. Print the last 3 failed requests, showing only time and user.
grep "FAIL" access.log  | tail -3 | awk -F' ' '{print $2, $3}'
# Q7. Which login shells appear in /etc/passwd, and how many accounts use each? (field 7, separator :)
awk -F':' '{print $7}' /etc/passwd | sort | uniq -c | sort -rn
# Q8. PREDICT, then explain: why do ls /etc | wc -l and ls -l /etc | wc -l differ by exactly one?
# Because ls -l adds an additional line "total 1160", which is total block allocation for the directory, and wc -l also counts it
# Bonus:
grep -E ':[0-5]0.*FAIL' access.log
grep -E ':[0-5]0.*FAIL' access.log | wc -l # for total
```

```
vboxuser@Ubuntu2604:~/os-lab3$ bash oneliners.sh
100
4: /page0 /page1 /page2 /page3
    125 /page0
    125 /page1
    125 /page2
    125 /page3
     15 user5 FAIL
     15 user3 FAIL
     14 user6 FAIL
     14 user4 FAIL
     14 user2 FAIL
     14 user1 FAIL
     14 user0 FAIL
     57 user3 OK
     15 user3 FAIL
Total requests: 72
10:10 user0
10:15 user5
10:20 user3
     40 /usr/sbin/nologin
      6 /bin/false
      2 /bin/bash
      1 /bin/sync
vboxuser@Ubuntu2604:~/os-lab3$
```

Figure 4.1: oneliners.sh

```
vboxuser@Ubuntu2604:~/os-lab3$ grep FAIL access.log | cut -d' ' -f4 | wc -w
100
vboxuser@Ubuntu2604:~/os-lab3$ awk '{print $5}' access.log | sort -u | awk '{pages = pages " " $1} END {print
 NR ":" pages}'
4: /page0 /page1 /page2 /page3
vboxuser@Ubuntu2604:~/os-lab3$ awk -F' ' '{print $5}' access.log | sort | uniq -c
    125 /page0
    125 /page1
    125 /page2
    125 /page3
vboxuser@Ubuntu2604:~/os-lab3$ awk '$4 != "OK" {print $3, $4}' access.log | sort | uniq -c | sort -rn
     15 user5 FAIL
     15 user3 FAIL
     14 user6 FAIL
     14 user4 FAIL
     14 user2 FAIL
     14 user1 FAIL
     14 user0 FAIL
vboxuser@Ubuntu2604:~/os-lab3$ awk '$3 == "user3" {print $3, $4}' access.log | sort | uniq -c | sort -rn | aw
k '{print $0; sum += $1} END {print "Total requests:", sum}'
     57 user3 OK
     15 user3 FAIL
Total requests: 72
vboxuser@Ubuntu2604:~/os-lab3$ grep "FAIL" access.log  | tail -3 | awk -F' ' '{print $2, $3}'
10:10 user0
10:15 user5
10:20 user3
vboxuser@Ubuntu2604:~/os-lab3$ awk -F':' '{print $7}' /etc/passwd | sort | uniq -c | sort -rn
     40 /usr/sbin/nologin
      6 /bin/false
      2 /bin/bash
      1 /bin/sync
```

Figure 4.2: All commands run separately

```
vboxuser@Ubuntu2604:~/os-lab3$ grep -E ':[0-5]0.*FAIL' access.log
2026-10-05 10:10 user3 FAIL /page2
2026-10-05 10:20 user6 FAIL /page0
2026-10-05 10:30 user2 FAIL /page2
2026-10-05 10:40 user5 FAIL /page0
2026-10-05 10:50 user1 FAIL /page2
2026-10-05 10:00 user4 FAIL /page0
2026-10-05 10:10 user0 FAIL /page2
2026-10-05 10:20 user3 FAIL /page0
2026-10-05 10:30 user6 FAIL /page2
2026-10-05 10:40 user2 FAIL /page0
2026-10-05 10:50 user5 FAIL /page2
2026-10-05 10:00 user1 FAIL /page0
2026-10-05 10:10 user4 FAIL /page2
2026-10-05 10:20 user0 FAIL /page0
2026-10-05 10:30 user3 FAIL /page2
2026-10-05 10:40 user6 FAIL /page0
2026-10-05 10:50 user2 FAIL /page2
2026-10-05 10:00 user5 FAIL /page0
2026-10-05 10:10 user1 FAIL /page2
2026-10-05 10:20 user4 FAIL /page0
2026-10-05 10:30 user0 FAIL /page2
2026-10-05 10:40 user3 FAIL /page0
2026-10-05 10:50 user6 FAIL /page2
2026-10-05 10:00 user2 FAIL /page0
2026-10-05 10:10 user5 FAIL /page2
2026-10-05 10:20 user1 FAIL /page0
2026-10-05 10:30 user4 FAIL /page2
2026-10-05 10:40 user0 FAIL /page0
2026-10-05 10:50 user3 FAIL /page2
2026-10-05 10:00 user6 FAIL /page0
2026-10-05 10:10 user2 FAIL /page2
2026-10-05 10:20 user5 FAIL /page0
2026-10-05 10:30 user1 FAIL /page2
2026-10-05 10:40 user4 FAIL /page0
2026-10-05 10:50 user0 FAIL /page2
2026-10-05 10:00 user3 FAIL /page0
2026-10-05 10:10 user6 FAIL /page2
2026-10-05 10:20 user2 FAIL /page0
2026-10-05 10:30 user5 FAIL /page2
2026-10-05 10:40 user1 FAIL /page0
2026-10-05 10:50 user4 FAIL /page2
2026-10-05 10:00 user0 FAIL /page0
2026-10-05 10:10 user3 FAIL /page2
2026-10-05 10:20 user6 FAIL /page0
2026-10-05 10:30 user2 FAIL /page2
2026-10-05 10:40 user5 FAIL /page0
2026-10-05 10:50 user1 FAIL /page2
2026-10-05 10:00 user4 FAIL /page0
2026-10-05 10:10 user0 FAIL /page2
2026-10-05 10:20 user3 FAIL /page0
vboxuser@Ubuntu2604:~/os-lab3$ grep -E ':[0-5]0.*FAIL' access.log | wc -l
50
```

Figure 4.3: Bonus task

Part 5. First script

```
#!/bin/bash
# usage: ./report.sh access.log log=$1
if [ ! -f "$log" ]; then
    echo "no such file: $log" >&2
    exit 1
fi
total=$(wc -l < "$log") fails=$(grep -c FAIL "$log")
echo "$total requests, $fails failed"
# TODO: for each of user0 ... user6, print "userN: X failures"
```

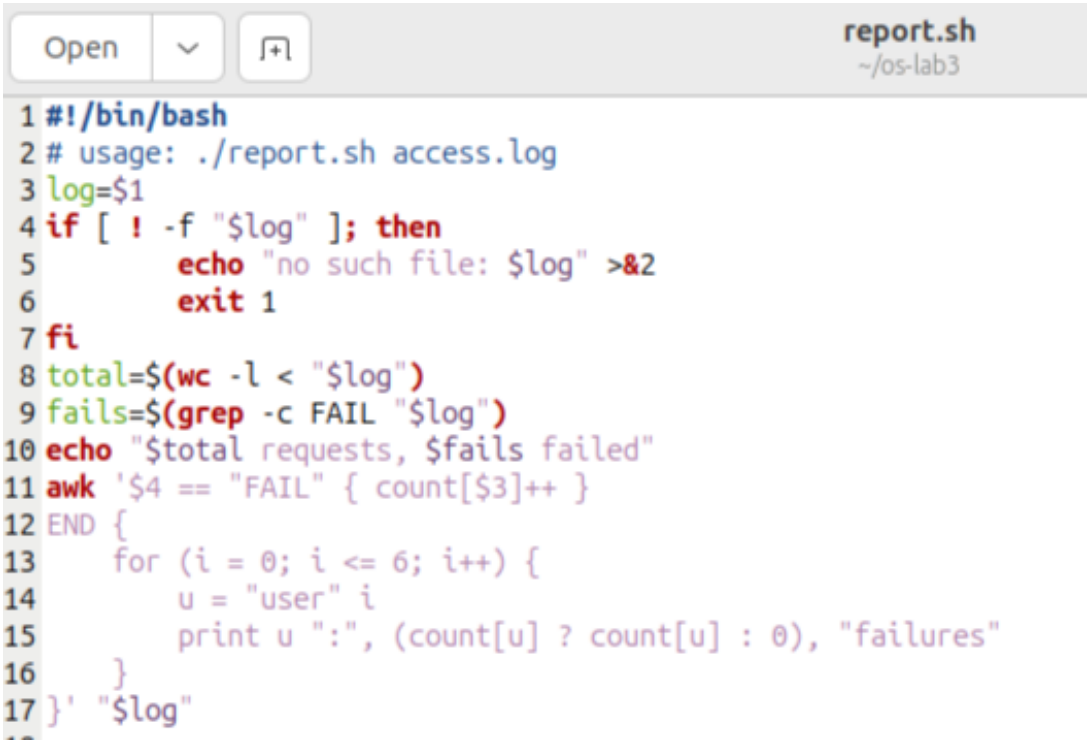


Figure 5.1: Full script

```
chmod +x report.sh
./report.sh access.log
./report.sh nope.log ; echo "exit code: $?"
```

```
vboxuser@Ubuntu2604:~/os-lab3$ chmod +x report.sh
vboxuser@Ubuntu2604:~/os-lab3$ ./report.sh access.log
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures
vboxuser@Ubuntu2604:~/os-lab3$ ./report.sh nope.log ; echo "exit code: $?"
no such file: nope.log
exit code: 1
vboxuser@Ubuntu2604:~/os-lab3$
```

Figure 5.2: Running the script

1. **What does the first line `#!/bin/bash` do, and what happens if you run `./report.sh` before `chmod +x` ?**
   The first line `#!/bin/bash` is a shebang, and it tells the kernel which interpreter should run the script, here bash. Before `chmod +x`, running `./report.sh` gives "Permission denied", because the file does not have execute permission yet.

2. **Why must you type `./report.sh` and not just `report.sh` ?**
   If we don't type the dot and slash, the shell will look in $PATH, but won't be able to find it, and `report.sh` will give "command not found". Typing with the dot and slash, we give the explicit path to the file in our current directory, so the shell runs it from the current directory instead of searching $PATH.

3. **Why does the script write the error with >&2 and end with exit 1?**
   The script writes the error with >&2 so that it goes to standard error (fd 2) rather than standard output, and it keeps the error messages separate from normal output. exit 1 stops the script right away and returns a nonzero code, which tells the caller (and operators like &&, ||, and $?) that the script failed. Exit code 0 means success; anything else means failure.

CONCLUSION

In this laboratory work I explored the Linux command line and learned how the shell processes a command before it runs, how data flows between programs, and how simple tools can be combined to answer real questions about data.

In the first part I learned to work faster in bash using Tab completion and history search (Ctrl+R), and to find help with man and help. I saw that some commands are shell built-ins while others are external programs found through the directories in $PATH. cd must be a built-in because the working directory is a property of the shell process itself: an external program would run in a child process and could not change its parent's directory. I also saw that a command which is not found returns exit code 127.

In the second part I saw that the shell, not the command, performs expansions such as globbing (*, ?, [a-b]), brace expansion, variable expansion and command substitution, before the command starts. For this reason echo *.c printed file names even though echo knows nothing about files: it simply received a.c b.c as arguments. I also learned how quoting changes this behavior: single quotes disable all expansion, double quotes still allow $ expansion, and a backslash escapes a single character. The failure of ls alfa beta.txt showed that an unquoted space splits a name into two arguments, which can be fixed by quoting the name or escaping the space.

In the third part I worked with standard streams (file descriptors 0, 1 and 2) and redirection with >, >>, <, 2> and 2>&1. Comparing > all.txt 2>&1 with 2>&1 > all.txt showed that redirections are processed from left to right, so their order changes the result. I also learned why wc -l < access.log prints only a number (the program does not receive the file name), and how the operators ;, && and ||chain commands depending on their exit codes, and how & and jobs manage background tasks.

In the fourth part I used filters (grep, cut, sort, uniq, wc, head, tail) joined by pipes to answer questions about the 500-line access log. Small tools that each do one job become powerful when combined, and the totals can be cross-checked (for example, the per-page counts add up to 500 and the per-user failure counts add up to the total number of failures).

In the last part I turned these commands into a script, report.sh. I learned that the shebang line #!/bin/bash tells the kernel which interpreter to use, that the file needs execute permission (chmod +x), and that it is started as ./report.sh because the current directory is not in $PATH. The script checks its input, writes errors to stderr with >&2, and returns a nonzero exit code with exit 1, so that other programs can detect failure. The output of the script on access.log (500 requests, 100 failed, with 14 or 15 failures per user) agrees with the one-liners from Part 4.

Overall, the main lesson of this lab is that the shell is a powerful layer between the

user and the programs: it expands, redirects and connects commands, and understanding the order in which it does this makes it possible to predict the result of a command line and to build useful tools from simple parts.

REFERENCES

1. https://github.com/mit-pdos
2. https://pdos.csail.mit.edu/6.828/2023/xv6/book-riscv-rev3.pdf
3. https://www.geeksforgeeks.org/linux-unix/difference-between-single-and-double-quotes-in-shell-script-and-linux/
4. https://www.geeksforgeeks.org/linux-unix/file-globbing-linux/
5. https://www.geeksforgeeks.org/linux-unix/sh-file-format/
6. https://www.geeksforgeeks.org/linux-unix/what-does-chmod-x-do-and-how-to-use-it/